claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-a6b6e803da5ada06a.jsonl`
- SHA-256: `64e839b9b8c331e478b2ee2608deeeddbc5d2e83dc2a9371f3945913336a13fc`
- Source modified: `2026-06-25T16:57:05+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T16:54:30.951Z)

PR #380 "feat(O1/P11): Gemini remote-file GC ? startup orphan sweep" is CHECKED OUT (branch o1-p11/gemini-remote-file-gc). It adds a best-effort startup sweep that deletes orphaned Gemini File-API uploads after a hard crash. Changed files (read the real diff with: git diff origin/master...HEAD): backend/providers/gemini.py (new method GeminiProvider.cleanup_orphaned_files(grace_sec=3600) using client.files.list()/delete()); backend/main.py (new _maybe_sweep_gemini_orphans(config), wired in main() next to fail_stale_jobs()); tests/test_provider_gemini.py (+6 tests). CI is GREEN ? do NOT re-run CI/ruff/mypy/tests; review the CODE for correctness, safety, and conventions. Read the actual files + surrounding code to ground every claim.

Review ONLY the 6 new tests in tests/test_provider_gemini.py. Does each assert the REAL behavior (stale-video-only delete incl. keep-cases fresh/non-video/no-timestamp; list-failure?0 swallowed; per-file-delete-failure swallowed + NOT counted; no-op for non-gemini and for no-key; sweep-runs-and-swallows)? GAPS: untested branches (pagination, timezone edge, the exact grace boundary, multiple stale files, the create_time-comparison path)? Are the mocks faithful to the real google-genai files.list()/files.delete() shape (attribute names, return types)? Report findings + fix.

2. assistant (2026-06-25T16:54:34.381Z)

I'll review the 6 new tests in the PR. Let me start by reading the actual diff and the relevant files.

3. user (2026-06-25T16:54:35.566Z)

```
 backend/main.py               |  25 ++++++++++
 backend/providers/gemini.py   |  43 ++++++++++++++++
 tests/test_provider_gemini.py | 104 ++++++++++++++++++++++++++++++++++++++++++
 3 files changed, 172 insertions(+)
```

4. assistant (2026-06-25T16:54:37.790Z)

Let me read the full diff and the test file, plus the real implementation.

5. user (2026-06-25T16:54:38.404Z)

```
diff --git a/backend/main.py b/backend/main.py
index 5561cee..ea5a875 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -64,6 +64,7 @@ from backend.pipeline.segmenters.base import segmenter_registry
 from backend.pipeline.stitcher.compiler import VideoCompiler
 from backend.pipeline.stitcher.watermark_i18n import localize_watermark
```

```
 from backend.pipeline.validators.base import registry
+from backend.providers import provider_registry
 from backend.reporting import emit_indexer_report
 from backend.results import Failure  # standardized error/result contract
 from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
@@ -1184,6 +1185,25 @@ def _enforce_startup_or_exit(config) -> None:
         sys.exit(1)


+def _maybe_sweep_gemini_orphans(config) -> None:
+    """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed run.
+    A no-op unless the configured provider is Gemini AND its API key is present. Never raises ?
+    a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
tests."""
+    try:
+        provider_name = getattr(config, "ai_provider", "")
+        if provider_name != "gemini":
+            return
+        api_key = os.environ.get("GEMINI_API_KEY")
+        if not api_key:
+            return
+        provider = provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)
+        deleted = provider.cleanup_orphaned_files()
+        if deleted:
+            logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted)
+    except Exception as e:
+        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
+
+
 def main():
     parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
     parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
@@ -1319,6 +1339,11 @@ def main():
     if swept:
         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

+    # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
+    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
+    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
+    _maybe_sweep_gemini_orphans(global_config)
+
     # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
     if args.video_path and not args.server and not args.app:
         print(f"[CLI] Processing video file: {args.video_path}")
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
 import logging
 import os
 import time
+from datetime import datetime, timedelta, timezone
 from typing import Tuple

 from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
         except Exception:
             pass

+    def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+        """Sweep and delete video files orphaned on the Gemini File API by a previous
+        crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+        leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
```

```
+        that reach a ``finally``/except).
+
+        Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+        creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+        Never raises; returns the count deleted. Intended to run once at startup.
+
+        Args:
+            grace_sec: only delete files older than this (default 1h). Protects a file a
+                live worker may still be processing.
+        """
+        cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+        deleted = 0
+        try:
+            files = list(self.client.files.list())
+        except Exception as e:
+            logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+            return 0
+        for f in files:
+            try:
+                mime = getattr(f, "mime_type", "") or ""
+                if not mime.startswith("video/"):
+                    continue
+                # f.create_time is an RFC3339 string on the File API.
+                ct = getattr(f, "create_time", None)
+                if ct is None:
+                    # No timestamp ? can't prove it's stale; leave it (safe default).
+                    continue
+                created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+                if created > cutoff:
+                    continue  # too recent ? a live worker may own it
+                self.client.files.delete(name=f.name)
+                deleted += 1
+            except Exception as e:
+                logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+        if deleted:
+            logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
+        return deleted
+
     def analyze_video(self, video_path: str, prompt: str,
                       chunk_duration: float = 0.0,
                       label: str = "") -> Tuple[list, dict]:
diff --git a/tests/test_provider_gemini.py b/tests/test_provider_gemini.py
index 141af81..5df2ef4 100644
--- a/tests/test_provider_gemini.py
+++ b/tests/test_provider_gemini.py
@@ -101,3 +101,107 @@ def test_gemini_provider_upload_fail():
         provider = GeminiProvider(api_key="dummy_api_key", model_name="gemini-2.5-flash")
         segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
         assert len(segments) == 0
+
+
+
+# --- O1/P11: Gemini remote-file GC (startup orphan sweep) ---
+
+def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
+    f = MagicMock()
+    f.name = name
+    f.mime_type = mime
+    f.create_time = create_time_iso
+    f.state = MagicMock()
+    f.state.name = state
+    return f
+
+
```

```
+def test_cleanup_orphaned_files_deletes_stale_videos_only():
+    """Deletes video files older than the grace window; leaves non-videos and fresh files."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
+    files = [
+        _file_mock("old-vid", "video/mp4", stale),       # stale video -> deleted
+        _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
+        _file_mock("old-img", "image/png", stale),        # non-video -> kept
+        _file_mock("notts", "video/mp4", None),           # no timestamp -> kept (safe)
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1
+    provider._client.files.delete.assert_called_once_with(name="old-vid")
+
+
+def test_cleanup_orphaned_files_list_failure_is_swallowed():
+    """A list() failure must not raise ? the sweep is best-effort and must not block
startup."""
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.side_effect = RuntimeError("network")
+        assert provider.cleanup_orphaned_files() == 0
+
+
+def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
+    """A delete() failure on one file must not abort the sweep."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    files = [
+        _file_mock("a", "video/mp4", stale),
+        _file_mock("b", "video/mp4", stale),
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        # First delete raises, second succeeds ? _safe_delete swallows it.
+        provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1  # only the second file counted
+
+
+
+# --- O1/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
+
+def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
+    """A non-Gemini provider must short-circuit (no client built, no sweep)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "openai"
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == []  # provider never instantiated
+
+
+def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
```

```
+    """No GEMINI_API_KEY -> no sweep (can't auth)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    monkeypatch.delenv("GEMINI_API_KEY", raising=False)
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == []
+
+
+def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
+    """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
startup)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    cfg.ai_model = "gemini-2.5-flash"
+    provider = MagicMock()
+    provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
+    monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    # Must not raise.
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    provider.cleanup_orphaned_files.assert_called_once()
```

6. assistant (2026-06-25T16:54:41.138Z)

Now let me look at the real implementation file and the `_client`/`client` attribute relationship, plus the test file header for imports.

7. user (2026-06-25T16:54:41.725Z)

```
1       import json
2       import logging
3       import os
4       import time
5       from datetime import datetime, timedelta, timezone
6       from typing import Tuple
7
8       from google import genai
9       from google.genai import types
10
11      from backend.infrastructure.execution_policy import (
12          ExecutionPolicy,
13          PolicyTimeout,
14          poll_until,
15          run_bounded,
16      )
17      from backend.providers.base import AIVideoProvider
18
19      logger = logging.getLogger(__name__)
20
21      # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
22      # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
23      MAX_UPLOAD_MB = 500
24
25      class GeminiProvider(AIVideoProvider):
26          def __init__(self, api_key: str, model_name: str):
27              super().__init__(api_key, model_name)
28              # Initialize client lazily to avoid connection/auth issues during class
instantiation
29              self._client = None
```

```
30
31          @property
32          def client(self):
33              if self._client is None:
34                  self._client = genai.Client(api_key=self.api_key)
35              return self._client
36
37          def _safe_delete(self, video_file) -> None:
38              """Best-effort cleanup of an uploaded File (never raises)."""
39              try:
40                  self.client.files.delete(name=video_file.name)
41              except Exception:
42                  pass
43
44          def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
45              """Sweep and delete video files orphaned on the Gemini File API by a previous
46              crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
47              leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
48              that reach a ``finally``/except).
49
50              Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
whose
51              creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
upload).
52              Never raises; returns the count deleted. Intended to run once at startup.
53
54              Args:
55                  grace_sec: only delete files older than this (default 1h). Protects a file a
56                      live worker may still be processing.
57              """
58              cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
59              deleted = 0
60              try:
61                  files = list(self.client.files.list())
62              except Exception as e:
63                  logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
%s", e)
64                  return 0
65              for f in files:
66                  try:
67                      mime = getattr(f, "mime_type", "") or ""
68                      if not mime.startswith("video/"):
69                          continue
70                      # f.create_time is an RFC3339 string on the File API.
71                      ct = getattr(f, "create_time", None)
72                      if ct is None:
73                          # No timestamp ? can't prove it's stale; leave it (safe default).
74                          continue
75                      created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
76                      if created > cutoff:
77                          continue  # too recent ? a live worker may own it
78                      self.client.files.delete(name=f.name)
79                      deleted += 1
80                  except Exception as e:
81                      logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
82              if deleted:
83                  logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
84              return deleted
85
86          def analyze_video(self, video_path: str, prompt: str,
87                            chunk_duration: float = 0.0,
88                            label: str = "") -> Tuple[list, dict]:
89              log_prefix = f"Gemini {label}" if label else "Gemini"
90
```

```
91              metrics: dict = {
92                  "upload_time": 0.0,
93                  "process_time": 0.0,
94                  "gen_time": 0.0,
95              }
```

8. assistant (2026-06-25T16:54:43.821Z)

Now let me read the test file from the top to understand the imports used.

9. user (2026-06-25T16:54:44.644Z)

```
1       import logging
2       from unittest.mock import MagicMock, patch
3
4       from backend.providers.gemini import GeminiProvider
5
6
7       def test_gemini_provider_generate_failure_is_flagged():
8           """A generate() give-up (per-attempt timeout / 503 burst after retries) must set
9           metrics['generate_failed'] so a FAILED chunk isn't silently mistaken for 'no
rallies'
10          (the silent-partial-result guard the segmenter surfaces loudly)."""
11          with patch("backend.providers.gemini.genai") as mock_genai, \
12               patch("backend.providers.gemini.run_bounded", side_effect=RuntimeError("503
exhausted")):
13              client_mock = MagicMock()
14              mock_genai.Client.return_value = client_mock
15              file_mock = MagicMock()
16              file_mock.state.name = "ACTIVE"
17              file_mock.name = "dummy_file"
18              client_mock.files.upload.return_value = file_mock
19              client_mock.files.get.return_value = file_mock
20
21              provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
22              segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)
23
24              assert segments == []                       # no analysis for this chunk
25              assert metrics.get("generate_failed") is True   # ...but the failure is FLAGGED,
not silent
26
27
28      def test_gemini_provider_success():
29          # Mock GenAI client
30          with patch("backend.providers.gemini.genai") as mock_genai:
```

10. assistant (2026-06-25T16:54:47.318Z)

Now let me verify the real shape of the google-genai `files.list()`/`files.delete()` API and the `create_time` type. Let me check whether the SDK is installed and inspect the actual types.

11. user (2026-06-25T16:54:50.302Z)

```
['_abc_impl', '_calculate_keys', '_check_field_type_mismatches', '_copy_and_set_values',
'_from_response', '_get_value', '_iter', '_setattr_handler', 'construct', 'copy', 'dict',
'from_orm', 'json', 'model_computed_fields', 'model_config', 'model_construct', 'model_copy',
'model_dump', 'model_dump_json', 'model_extra', 'model_fields', 'model_fields_set',
'model_json_schema', 'model_parametrized_name', 'model_post_init', 'model_rebuild',
'model_validate', 'model_validate_json', 'model_validate_strings', 'parse_file', 'parse_obj',
'parse_raw', 'schema', 'schema_json', 'to_json_dict', 'update_forward_refs', 'validate']
```

12. user (2026-06-25T16:54:55.604Z)

```
name -> typing.Optional[str]
mime_type -> typing.Optional[str]
create_time -> typing.Optional[datetime.datetime]
state -> typing.Optional[google.genai.types.FileState]
update_time -> typing.Optional[datetime.datetime]
expiration_time -> typing.Optional[datetime.datetime]
```

13. assistant (2026-06-25T16:54:58.604Z)

```
This is a significant finding. The real `create_time` is a `datetime.datetime` object, NOT an
RFC3339 string. Let me verify how `files.list()` returns data (pager type) and confirm the
`delete` signature.
```

14. user (2026-06-25T16:55:01.920Z)

```
--- list signature ---
(self, *, config: Union[google.genai.types.ListFilesConfig,
google.genai.types.ListFilesConfigDict, NoneType] = None) ->
google.genai.pagers.Pager[google.genai.types.File]
--- delete signature ---
(self, *, name: str, config: Union[google.genai.types.DeleteFileConfig,
google.genai.types.DeleteFileConfigDict, NoneType] = None) ->
google.genai.types.DeleteFileResponse
```

15. assistant (2026-06-25T16:55:07.468Z)

```
Now I've confirmed the real API shape. Key facts:
- `files.list()` returns a `Pager[File]` (iterable, supports pagination automatically when
iterated).
- `files.delete(name=...)` ? keyword-only `name`. Matches.
- `File.create_time` is a `datetime.datetime` object, NOT an RFC3339 string.

Let me verify the production code's handling of a real `datetime` object via
`str(ct).replace(...)` and `fromisoformat`, to confirm the test/prod mismatch matters.
```

16. user (2026-06-25T16:55:09.642Z)

```
repr: datetime.datetime(2026, 6, 25, 14, 55, 9, 613455, tzinfo=datetime.timezone.utc)
str: 2026-06-25 14:55:09.613455+00:00
parsed OK: 2026-06-25 14:55:09.613455+00:00
aware: True
```

17. assistant (2026-06-25T16:55:16.782Z)

```
So `str(datetime)` produces a space-separated form which `fromisoformat` happens to accept
(Python 3.11+ is lenient; even 3.8 `fromisoformat` accepts the space separator since it's the
`isoformat(sep=' ')` round-trip form). So prod works on a real datetime, but the tests feed
`.isoformat()` strings ? they do NOT exercise the real datetime-object path. Let me confirm the
naive-datetime edge (no tzinfo) which is a real risk if the SDK ever returns naive.
```

18. user (2026-06-25T16:55:19.455Z)

```
<string>:4: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes in UTC:
datetime.datetime.now(datetime.UTC).
naive parsed: 2026-06-25 16:55:19.433582 tzinfo: None
COMPARE ERROR: TypeError can't compare offset-naive and offset-aware datetimes
```

19. assistant (2026-06-25T16:55:25.950Z)

```
Confirmed: a naive `create_time` (or naive ISO string without offset) raises `TypeError` on the
comparison ? but that's caught by the per-file `except` in prod (logged at debug, file skipped).
Not a crash, but it means a naive-timestamped stale file would be silently skipped and never
reclaimed. The tests don't cover this.
```

Now let me check one more critical thing: the test uses `provider._client = MagicMock()` directly, but the `cleanup_orphaned_files` code accesses `self.client` (the property). Let me confirm the property returns `_client` when it's already set (it does per line 33-35), so the mock is reached. Good.

Now let me verify the `delete.assert_called_once_with(name="old-vid")` ? but note `f.name` in the mock. There's a subtle MagicMock gotcha: `name` is a reserved kwarg on `MagicMock()`. Let me check whether `f.name = name` in `_file_mock` actually sets the attribute correctly.

20. user (2026-06-25T16:55:27.644Z)

```
name attr: old-vid
ctor name (reserved): <MagicMock name='foo.name' id='1444817973808'>
```

21. assistant (2026-06-25T16:55:38.960Z)

Good ? the helper sets `f.name = name` *after* construction (line in `_file_mock`), which correctly assigns the attribute. So `assert_called_once_with(name="old-vid")` is valid. The `_file_mock` helper avoids the reserved-kwarg trap correctly.

Now let me check the `state`/`state.name` field set in `_file_mock` ? it's set but never used by `cleanup_orphaned_files`. Minor: dead test scaffolding. Let me also confirm the comment on line about `_safe_delete` swallowing in the per-file-delete test is accurate (the prod code does NOT call `_safe_delete`; it catches inline).

Let me verify there are no other tests in the file referencing these and check the test that asserts `deleted == 1` for per-file failure precisely matches prod behavior (first delete raises ? not counted; second succeeds ? counted).

Looking at prod: the `self.client.files.delete(name=f.name)` is *inside* the try, and `deleted += 1` is *after* it. So if delete raises, `deleted += 1` is skipped ? correct, count is 1. The test comment says "_safe_delete swallows it" which is **inaccurate** ? prod does not use `_safe_delete` here; it uses the inline `except Exception`. Minor comment inaccuracy.

Let me now check the timezone/grace-boundary and pagination concerns more concretely, and confirm whether `iter(files)` faithfully represents the Pager (it's iterable, so yes for the happy path, but pagination across pages isn't exercised). Let me look at whether there's existing test convention for mocking the pager.

22. user (2026-06-25T16:55:42.534Z)

```
['config', 'name', 'next_page', 'page', 'page_size', 'sdk_http_response']
--- iter? --- True True
```

23. assistant (2026-06-25T16:55:48.944Z)

The `Pager` is iterable and transparently handles pagination during iteration, so `list(self.client.files.list())` in prod will pull all pages. The tests use `iter([...])` which is a faithful enough stand-in for *single-page* iteration but does not prove multi-page behavior. Since prod relies on the Pager's own pagination (not manual `next_page` calls), this is a low-severity gap ? there's no prod pagination code to test, only the SDK's. I'll note it as a NIT.

Now let me verify the claim in the task about what each test asserts, and check the exact-grace-boundary. The boundary is `if created > cutoff: continue` ? strictly greater means a file exactly at the cutoff (created == cutoff) would be deleted. The tests use 2h-stale and 5min-fresh, never near the 1h boundary. Untested boundary.

Let me do a final check: confirm the test file is otherwise structured well and that `monkeypatch.setattr(main_mod.provider_registry, "get", ...)` matches the real `provider_registry` import in main.py.

24. user (2026-06-25T16:55:52.091Z)

```
type: ProviderRegistry
has get: True
(name: str, api_key: str, model_name: str) -> backend.providers.base.AIVideoProvider
```

25. assistant (2026-06-25T16:55:59.972Z)

The wiring tests' `monkeypatch.setattr(main_mod.provider_registry, "get", ...)` is faithful ? `get(name, api_key, model_name)`. The non-gemini test asserts `called == []`, but note: in prod `_maybe_sweep_gemini_orphans` short-circuits BEFORE calling `provider_registry.get` for non-gemini and no-key cases, so `get` is never invoked ? the lambda's `called.append` correctly proves the short-circuit. Good.

One subtlety in `test_maybe_sweep_skips_non_gemini_provider`: the lambda captures kwargs only (`called.append(k)`), and `get` is called positionally in prod? Let me check the prod call: `provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)` ? `"gemini"` positional, rest kwargs. The lambda is `lambda *a, **k: called.append(k) or MagicMock()`, so it accepts both. But the assertion is just `called == []` (never called), so it doesn't matter for these two skip tests. Fine.

Now let me confirm there's no test that actually exercises the `create_time`-as-real-datetime path or asserts on the parsing ? this is the core faithfulness gap. The tests ALL pass `.isoformat()` strings, but the real SDK returns `datetime` objects. Let me verify the test would still pass if I substituted a real datetime (to confirm prod is robust) and that the gap is purely test-fidelity, not a prod bug.

26. user (2026-06-25T16:56:07.193Z)

```
deleted (real-datetime path): 1
PROD HANDLES REAL DATETIME: OK
```

27. assistant (2026-06-25T16:56:14.666Z)

Confirmed. Prod handles the real `datetime` object correctly (via `str()` ? space-sep ISO ? `fromisoformat`), but **no test exercises that real shape** ? all tests feed `.isoformat()` strings. This is the most important fidelity gap: if someone refactored the parsing to e.g. `datetime.fromisoformat(ct)` (assuming a string), the tests would still pass with string mocks but prod would crash on the real `datetime`. The tests don't lock in the actual contract.

Let me do a final scan of the whole new test block once more to ensure I've captured every assertion and didn't miss any claimed-but-absent coverage.

28. user (2026-06-25T16:56:15.780Z)

```
104
105
106
107     # --- O1/P11: Gemini remote-file GC (startup orphan sweep) ---
108
109     def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
110         f = MagicMock()
111         f.name = name
112         f.mime_type = mime
113         f.create_time = create_time_iso
114         f.state = MagicMock()
115         f.state.name = state
116         return f
117
118
119     def test_cleanup_orphaned_files_deletes_stale_videos_only():
120         """Deletes video files older than the grace window; leaves non-videos and fresh
files."""
121         from datetime import datetime, timedelta, timezone
122         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
123         fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
```

```
        files = [
            _file_mock("old-vid", "video/mp4", stale),       # stale video -> deleted
            _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
            _file_mock("old-img", "image/png", stale),        # non-video -> kept
            _file_mock("notts", "video/mp4", None),           # no timestamp -> kept (safe)
        ]
        with patch("backend.providers.gemini.genai"):
            provider = GeminiProvider(api_key="k", model_name="m")
            provider._client = MagicMock()
            provider._client.files.list.return_value = iter(files)
            deleted = provider.cleanup_orphaned_files(grace_sec=3600)
        assert deleted == 1
        provider._client.files.delete.assert_called_once_with(name="old-vid")


    def test_cleanup_orphaned_files_list_failure_is_swallowed():
        """A list() failure must not raise ? the sweep is best-effort and must not block
startup."""
        with patch("backend.providers.gemini.genai"):
            provider = GeminiProvider(api_key="k", model_name="m")
            provider._client = MagicMock()
            provider._client.files.list.side_effect = RuntimeError("network")
            assert provider.cleanup_orphaned_files() == 0


    def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
        """A delete() failure on one file must not abort the sweep."""
        from datetime import datetime, timedelta, timezone
        stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
        files = [
            _file_mock("a", "video/mp4", stale),
            _file_mock("b", "video/mp4", stale),
        ]
        with patch("backend.providers.gemini.genai"):
            provider = GeminiProvider(api_key="k", model_name="m")
            provider._client = MagicMock()
            provider._client.files.list.return_value = iter(files)
            # First delete raises, second succeeds ? _safe_delete swallows it.
            provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
            deleted = provider.cleanup_orphaned_files(grace_sec=3600)
        assert deleted == 1  # only the second file counted


    # --- O1/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---

    def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
        """A non-Gemini provider must short-circuit (no client built, no sweep)."""
        import backend.main as main_mod
        cfg = MagicMock()
        cfg.ai_provider = "openai"
        called = []
        monkeypatch.setattr(main_mod.provider_registry, "get",
                            lambda *a, **k: called.append(k) or MagicMock())
        monkeypatch.setenv("GEMINI_API_KEY", "k")
        main_mod._maybe_sweep_gemini_orphans(cfg)
        assert called == []  # provider never instantiated


    def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
        """No GEMINI_API_KEY -> no sweep (can't auth)."""
        import backend.main as main_mod
        cfg = MagicMock()
        cfg.ai_provider = "gemini"
        monkeypatch.delenv("GEMINI_API_KEY", raising=False)
```

```
188         called = []
189         monkeypatch.setattr(main_mod.provider_registry, "get",
190                             lambda *a, **k: called.append(k) or MagicMock())
191         main_mod._maybe_sweep_gemini_orphans(cfg)
192         assert called == []
193
194
195     def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
196         """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
startup)."""
197         import backend.main as main_mod
198         cfg = MagicMock()
199         cfg.ai_provider = "gemini"
200         cfg.ai_model = "gemini-2.5-flash"
201         provider = MagicMock()
202         provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
203         monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
204         monkeypatch.setenv("GEMINI_API_KEY", "k")
205         # Must not raise.
206         main_mod._maybe_sweep_gemini_orphans(cfg)
207         provider.cleanup_orphaned_files.assert_called_once()
```

29. assistant (2026-06-25T16:56:23.869Z)

I now have a complete picture. Let me note one more thing about `test_maybe_sweep_skips_when_no_api_key`: if the host machine running the test has a real `GEMINI_API_KEY` set, `monkeypatch.delenv(..., raising=False)` correctly removes it for the test scope, so the test is hermetic. Good.

Also, `test_maybe_sweep_runs_and_swallows_errors` is the *only* test proving the happy-path wiring actually calls `cleanup_orphaned_files`, and it does so via the error path. There's no test asserting the success log/positive-count happy path through `_maybe_sweep_gemini_orphans` (e.g. `deleted=2` ? `logger.info` "reclaimed"). Minor.

I have enough to report. Let me compile findings.

The task's premise checks out for the 6 tests' basic claims, but there are real fidelity gaps. The single most important one: **none of the tests use a real `datetime` object for `create_time`** ? all feed `.isoformat()` strings, while the live SDK returns `datetime.datetime`. This means the `str(ct).replace("Z",...)`/`fromisoformat` parsing ? the actual code under test ? is exercised against the *wrong input type*, so the tests would not catch a refactor that breaks the real contract.

30. user (2026-06-25T16:56:53.183Z)

Structured output provided successfully

31. assistant (2026-06-25T16:57:05.819Z)

I've completed the review of the 6 new tests in `tests/test_provider_gemini.py` against the real implementation and the actual google-genai API shape.

Summary of what I verified and found:

The 6 tests do assert their stated behavior ? stale-video-only deletion (with fresh/non-video/no-timestamp kept), list-failure?0, per-file-delete-failure swallowed and not counted, non-gemini and no-key short-circuits, and sweep-runs-and-swallows. The `_file_mock` helper correctly avoids the `MagicMock(name=...)` reserved-kwarg trap (sets `f.name` after construction), and the `provider_registry.get` and `files.delete(name=...)` mocks match the real signatures I confirmed via the installed SDK.

The most important gap (MEDIUM): the live SDK returns `File.create_time` as a real `datetime.datetime` (I confirmed: `Optional[datetime.datetime]`), but every test feeds an `.isoformat()` **string**. The actual parsing line under test

(`datetime.fromisoformat(str(ct).replace("Z","+00:00"))`) is therefore never exercised against the real input type. Prod happens to survive a real datetime (I ran it: deleted==1), so this is a test-fidelity gap, not a current prod bug ? but it means a refactor assuming a string would pass all tests and crash in production. The prod comment at `gemini.py:70` ("is an RFC3339 string") is also factually wrong.

Second (MEDIUM): a naive `create_time` raises `TypeError` on the tz-aware comparison (reproduced); it's swallowed at debug, so a genuinely-stale file is silently never reclaimed. Untested and arguably a prod hardening miss.

Plus untested grace-boundary / multiple-stale / non-Z-offset cases (LOW), an inaccurate `_safe_delete` comment, and no happy-path success/log assertion (NITs).

Relevant files (absolute):
- `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_provider_gemini.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\providers\gemini.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py`

Full findings with locations and suggested fixes are in the StructuredOutput.